

Le langage procédural en SQL sous PostGres Les fonctions

PL/pgSQL

(Procedural Language/postgreSQL Structured Query Language)

A decorative vertical pattern on the left side of the slide, composed of various blue and yellow hexagons of different sizes and orientations.

Avertissement

- Cette partie du cours n'est qu'un survol du langage PL/pgSQL, utile pour écrire des procédures stockées simples
- Elle laisse de côté de nombreuses fonctionnalités du langage

Introduction au PL/pgSQL

■ PL/pgSQL

- (*Procedural Language/PostgreSQL Structured Query Language*) est un langage procédural géré par PostgreSQL. Ce langage est très similaire au PL/SQL d'Oracle.

■ Pourquoi PL/pgSQL ?

- SQL est un langage non procédural
- Les traitements complexes sont parfois très difficile à écrire si on ne peut utiliser des variables et les structures de programmation comme les boucles et les alternatives
- On ressent vite le besoin d'un langage procédural pour lier plusieurs requêtes SQL avec des variables et dans les structures de programmation habituelles

A decorative graphic on the left side of the slide consisting of a vertical column of blue and yellow hexagons of various sizes, some overlapping.

Introduction

■ Principales caractéristiques de PL/pgSQL

- PL langage de programmation côté serveur
- Inspiré du PL/SQL d'Oracle
- Extension de SQL : des requêtes SQL cohabitent avec les structures de contrôle habituelles de la programmation structurée (blocs, alternatives, boucles)
- Des variables permettent l'échange d'information entre les requêtes SQL et le reste du programme

Introduction

■ Utilisation de PL/pgSQL

- ❑ PL/pgSQL peut être utilisé pour l'écriture des procédures stockées et des triggers
- ❑ Il convient aussi pour écrire des fonctions utilisateurs qui peuvent être utilisées dans les requêtes SQL (en plus des fonctions prédéfinies)

Structure d'un programme

- PL/pgSQL est un langage structuré en blocs. Le texte complet de la définition d'une fonction doit être un bloc. Un bloc est défini comme :

```
CREATE OR REPLACE FUNCTION nomfonction RETURNS type As $$  
DECLARE  
    déclarations de variables  
BEGIN  
    instructions ;  
END; $$ LANGUAGE plpgsql;
```

- Chaque déclaration et chaque expression au sein du bloc est terminée par un point-virgule
- les \$\$ permettent de délimiter la définition de la fonction (*sans cela le moteur SQL considérera le point virgule au sein du bloc comme une fin de requête SQL et générera une erreur*)

Erreur à ne pas commettre : voir la vidéo

The screenshot displays the phpPgAdmin web interface in a Mozilla Firefox browser window. The browser's address bar shows the URL: `https://a-nat.aix.univ-amu.fr/phpPgAdmin/sqledit.php?subject=table&server=local`. The interface is divided into several sections:

- Left Panel (Servers):** A tree view showing the database structure. Under "PostgreSQL", the "carmigna" database is selected, and its "Schémas" (Schemas) are listed, including `BD_TEST_OLAP`, `bd_gestion_vol`, `bd_gestion_vol1`, `bd_gestion_vol2`, `bd_salledebain`, `bd_stage`, `bd_voyage1`, `bdd_acapitainerie`, `ctrl1_bd_stage`, `ctrl1_bdd_car_net`, `ctrl1_formule1`, `ctrl2_stage`, `ctrl3_bd_biblio`, `ctrl3_cooperative`, `ctrl3_edt`, `ctrl3_stage`, `ctrl3_sucrerie`, `ctrl_bd_suivi_patient`, `ctrl_bdd_car_net_2020`, and `ctrl_edt`.
- Top Panel:** Displays "PostgreSQL 9.5.17 lancé" and "phpPgAdmin : Pos". It includes tabs for "Tables?", "Vues?", and "SQL?".
- Search Section:** Features a "Rechercher" (Search) button and a search bar. The "Serveur?" (Server?) dropdown is set to "PostgreSQL (localhost:5432:allow)", and the "Base de données?" (Database?) dropdown is set to "carmigna". The "Chemin de recherche du schéma?" (Schema search path?) field contains "pour test,public".
- SQL Editor:** A large text area for writing SQL queries, currently empty with a cursor.
- Buttons:** At the bottom, there are "Lancer" (Run) and "Réinitialiser" (Reset) buttons, along with a checkbox for "Pagner les résultats" (Show results).

Structure d'un programme

■ Exemple

■ définition de la fonction

```
CREATE OR REPLACE FUNCTION moySalaire() RETURNS NUMERIC AS $$  
    DECLARE  
        moyenne numeric;  
    BEGIN  
        SELECT AVG(salaire) INTO moyenne FROM client;  
        RETURN moyenne ;  
    END; $$  
LANGUAGE plpgsql;
```

■ *Exemple d'appel de la fonction*

```
SELECT moySalaire() ;
```

Les commentaires

- Les commentaires comme dans tout langage sont très utiles voir obligatoire!!!!

-- un commentaire sur une seule ligne

/ un commentaire sur plusieurs
lignes */*

Les variables

■ Règles d'écriture

■ Identificateurs PostGres :

- 60 caractères au plus ;
- doit commencer par une lettre ;
- peut contenir lettres, chiffres, _ , \$ et #
- insensible à la casse
- doivent être déclarées avant d'être utilisées

Les variables

■ Types des variables

- Les types habituels de Postgres : *integer*, *varchar*, *boolean*.....
- Types composites adaptés à la copie du type d'une colonne ou d'une ligne d'une table SQL : *%TYPE*, *%ROWTYPE*

■ La syntaxe générale d'une déclaration de variable est :

```
name [CONSTANT] type [NOT NULL] [{ DEFAULT | :=} expression];
```

- La clause DEFAULT, spécifie la valeur initiale si pas indiquée, initialisée à la valeur SQL NULL.
- L'option CONSTANT empêche l'assignation de la variable.

Les variables

■ Déclaration d'une variable

Doit être défini dans la partie DECLARE de la fonction

□ Exemples simples:

```
age integer;  
nom varchar(30);  
dateNaissance date;  
ok boolean := true;
```

□ Exemples avec affectation de valeur par défaut :

```
quantity integer DEFAULT 32 ;  
url varchar := 'http://mysite.com'  
user_id CONSTANT integer := 10 ;
```

!! Déclarations multiples interdites :
i, j integer;

Les variables

■ Déclaration %TYPE

On peut déclarer qu'une variable est du même type qu'une colonne d'une table ou d'une vue :

■ Syntaxe :

```
name tablename.columnname%TYPE;
```

■ Exemple :

```
NomEmp emp.name%TYPE;
```

*La variable **NomEmp** sera du même type que la colonne **name** de la table **emp***

Les variables

■ Déclaration %ROWTYPE

Une variable de type composite est appelée variable ligne (ou variable row-type) elle peut contenir une ligne entière d'une table.

■ Syntaxe générale :

```
name table_name%ROWTYPE;
```

■ Exemple :

```
employe emp%ROWTYPE;
```

■ *la variable employe est une structure contenant le nom et le type des colonnes de la table emp*

■ Les colonnes de la ligne (row) sont accessibles en utilisant la notation pointée, par exemple :

```
employe . name
```

Les variables

■ Type RECORD

■ Syntaxe :

nom RECORD ;

- Les variables record sont similaires aux variables de type ligne, mais n'ont pas de structure prédéfinie. Elles empruntent la structure effective de type ligne de la ligne à laquelle elles sont assignées durant une commande SELECT ou FOR. La sous-structure d'une variable record peut changer à chaque fois qu'on l'assigne. Une conséquence de cela est que jusqu'à ce qu'elle ait été assignée, elle n'a pas de sous-structure, et toutes les tentatives pour accéder à un de ses champs entraîneront une erreur d'exécution.

Les variables

■ Un exemple d'utilisation des types composites %ROWTYPE :

-- définition de la fonction

```
CREATE or replace FUNCTION name_form(p_id int) RETURNS text AS $$  
  DECLARE  
    form formateur%ROWTYPE;  
  BEGIN  
    SELECT * INTO form  
      FROM formateur  
      WHERE id_formateur=p_id ;  
    RETURN form.nom_formateur||' '|| form.prenom_formateur;  
  END; $$ LANGUAGE plpgsql;
```

-- appel de la fonction

```
select name_form(1);
```

laurent.carmignac@univ-amu.fr

Les variables

■ Un exemple d'utilisation des types composites **RECORD** :

-- définition de la fonction

```
CREATE or replace FUNCTION name_form(p_id int) RETURNS text AS $$  
  DECLARE  
    form RECORD;  
  BEGIN  
    SELECT nom_formateur, prenom_formateur INTO form  
      FROM formateur  
      WHERE id_formateur=p_id ;  
    RETURN form.nom_formateur||' '|| form.prenom_formateur;  
  END; $$ LANGUAGE plpgsql;
```

-- appel de la fonction

```
select name_form(1);
```

laurent.carmignac@univ-amu.fr

Les arguments d'une fonction

■ Introduction

- Vous pouvez utiliser tous les types classiques pour définir un argument
- Les arguments d'une fonction SQL peuvent être référencés dans le corps de la fonction en utilisant soit les noms soit les numéros.
- Si un argument est de type composite, la notation pointée, `nom_argument.nom_champ` ou `$1.nom_champ` peut être utilisé pour accéder aux attributs de l'argument de type composite.

Les arguments d'une fonction

■ Exemple

```
CREATE FUNCTION add_em(x integer, y integer) RETURNS integer
AS $$
Begin
    Return x + y;
End; $$ LANGUAGE SQL;
```

-- Ou avec le N° d'ordre de déclaration

```
CREATE FUNCTION add_em( integer, integer) RETURNS integer AS
$$
Begin
    Return $1 + $2;
End; $$ LANGUAGE SQL;
```

Les arguments d'une fonction

■ Alias de paramètres de fonctions

- Chaque paramètre anonyme peut être identifié par **\$n** où n est le numéro d'ordre du paramètre (voir diapo précédente)

■ Syntaxe :

nom ALIAS FOR **\$n**; *n*: le nième paramètre

■ Exemple :

```
CREATE or replace FUNCTION prixTTC(real, real) RETURNS real AS $$  
DECLARE  
    prixHT ALIAS FOR $1; -- alias sur le 1er paramètre de la fonction  
    taux ALIAS FOR $2; -- alias sur le 2ième paramètre de la fonction  
BEGIN  
    RETURN prixHT * (1+taux);  
END;  
$$ LANGUAGE plpgsql;
```

Les arguments d'une fonction

■ Fonctions avec des valeurs par défaut pour les arguments

- Les fonctions peuvent être déclarées avec des valeurs par défaut pour certains des paramètres en entrée ou pour tous.
- Les valeurs par défaut sont insérées quand la fonction est appelée avec moins d'arguments que à priori nécessaires.
- Comme les arguments peuvent seulement être omis à partir de la fin de la liste des arguments, tous les paramètres après un paramètre disposant d'une valeur par défaut disposeront eux-aussi d'une valeur par défaut.

Les arguments d'une fonction

■ Fonctions avec des valeurs par défaut pour les arguments

■ Par exemple :

```
CREATE FUNCTION somme (x numeric, y numeric default 2, z numeric default 2)
RETURNS numeric AS $$
BEGIN
    RETURN x + y + z ;
END; $$ LANGUAGE plpgsql;
```

Les appels suivants retournent :

```
SELECT somme (3, 5, 6) ;
```

somme
14

```
SELECT somme (3, 5) ;
```

somme
10

```
SELECT somme (3) ;
```

somme
7

ATTENTION avec le changement de type des arguments

: voir la vidéo

The screenshot displays the phpPgAdmin interface for PostgreSQL 9.5.17. On the left, a tree view shows the database structure, including tables, views, sequences, functions, and other objects. The 'pour test' database is selected. The main panel shows the 'Fonctions' (Functions) tab, indicating that no functions were found. A modal window is open, showing the 'Créer une fonction PL/SQL' (Create a PL/SQL function) form. The form includes fields for the server (PostgreSQL (localhost:5432:allow)), the database (carmigna), and the schema path (pour test,public). The SQL editor contains the following code:

```
CREATE OR REPLACE FUNCTION add(x real, y varchar) RETURNS NUMERIC AS $$  
DECLARE  
BEGIN  
    RETURN x + y ;  
END; $$  
LANGUAGE plpgsql;
```

At the bottom of the modal, there is a checkbox for 'Pager les résultats' (Pager the results) and two buttons: 'Lancer' (Run) and 'Réinitialiser' (Reset). The 'Haut de la page' (Top of the page) link is visible in the bottom right corner of the modal.

L'affectation

■ Affecter une valeur à une variable scalaire (une seule valeur)

- L'assignation d'une valeur à une variable ou à un champ row/record est écrite ainsi

identifiant := expression;

ou

identifiant = expression;

■ Exemple :

dateNaissance = '2004-10-10' ;

L'affectation

■ Affecter une valeur à une variable, valeur issue d'une requête

□ `SELECT source INTO destination .....`

Le résultat d'une commande `SELECT` manipulant plusieurs colonnes (**mais une seule ligne**) peut être assignée à une variable de type record ou ligne, ou une liste de valeurs scalaires.

```
SELECT col1, col2 INTO varcible1, varvarcible2
FROM ...;
```

Exemple :

```
DECLARE
```

```
  v_adr emp.empadr%TYPE; -- même type que la colonne empadr de la table emp
```

```
  v_prenom emp.empre%TYPE; -- même type que la colonne empre de la table emp
```

```
BEGIN
```

```
  SELECT emadr,empre INTO v_adr, v_prenom FROM emp
```

```
  WHERE empname like p_mynome;
```

.....

!! Attention un SELECT doit toujours avoir une variable qui reçoit le résultat

L'affectation

- Le résultat d'une commande SELECT manipulant une seule colonne (**mais une seule ligne**) peut être assignée à une variable scalaire.

identifiant = (la requête) ;

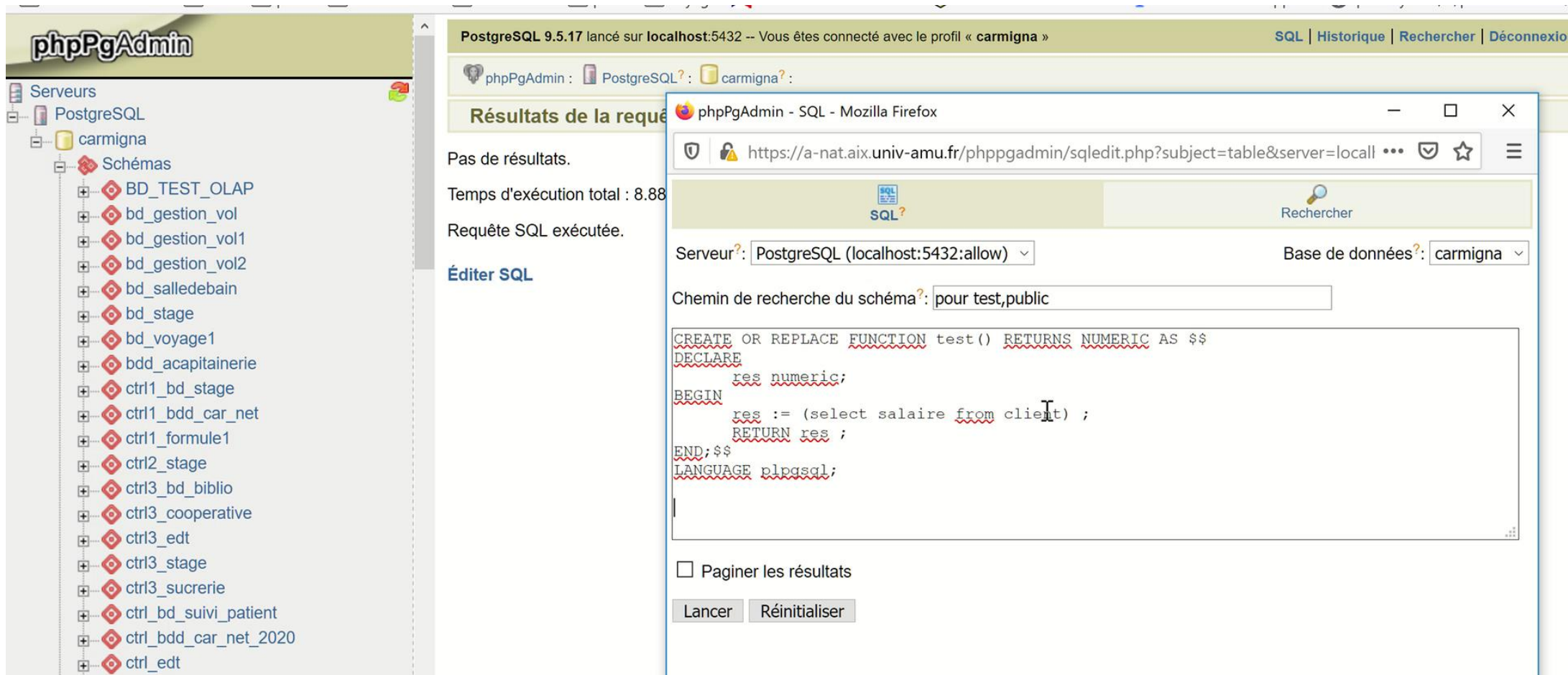
Ou

identifiant := (la requête) ;

Exemple :

```
v_prenom = (SELECT emprenom  
            FROM emp  
            WHERE id_emp=10) ;
```

Erreur à ne pas faire : : voir la vidéo



The screenshot displays the phpPgAdmin web interface for PostgreSQL 9.5.17 running on localhost:5432. The user is logged in as 'carmigna'. The left sidebar shows a tree view of databases and schemas, including 'BD_TEST_OLAP', 'bd_gestion_vol', 'bd_gestion_vol1', 'bd_gestion_vol2', 'bd_salledebain', 'bd_stage', 'bd_voyage1', 'bdd_acapitainerie', 'ctrl1_bd_stage', 'ctrl1_bdd_car_net', 'ctrl1_formule1', 'ctrl2_stage', 'ctrl3_bd_biblio', 'ctrl3_cooperative', 'ctrl3_edt', 'ctrl3_stage', 'ctrl3_sucrerie', 'ctrl_bd_suivi_patient', 'ctrl_bdd_car_net_2020', and 'ctrl_edt'.

The main panel shows the 'Résultats de la requête' (Query Results) section, indicating 'Pas de résultats.' (No results) and a 'Temps d'exécution total : 8.88' (Total execution time: 8.88). Below this, the 'Requête SQL exécutée.' (Executed SQL query) section is visible, with a link to 'Éditer SQL' (Edit SQL).

An overlay window titled 'phpPgAdmin - SQL - Mozilla Firefox' is open, showing the 'SQL' editor. The URL bar indicates the page is 'https://a-nat.aix.univ-amu.fr/phpPgAdmin/sqledit.php?subject=table&server=local'. The 'Serveur?' (Server?) dropdown is set to 'PostgreSQL (localhost:5432:allow)', and the 'Base de données?' (Database?) dropdown is set to 'carmigna'. The 'Chemin de recherche du schéma?' (Schema search path?) field contains 'pour test,public'. The SQL editor contains the following code:

```
CREATE OR REPLACE FUNCTION test() RETURNS NUMERIC AS $$  
DECLARE  
    res numeric;  
BEGIN  
    res := (select salaire from client) ;  
    RETURN res ;  
END; $$  
LANGUAGE plpgsql;
```

At the bottom of the editor window, there is a checkbox for 'Pager les résultats' (Pager the results) and two buttons: 'Lancer' (Run) and 'Réinitialiser' (Reset).

L'affectation

- Le résultat d'une requête utilisé dans un test conditionnel :

if (*la requête de comparaison*) *opérateur opérande*

- Exemple :

```
IF (SELECT nom_formateur FROM formateur
    WHERE id_formateur=p_id ) = 'LEGRAND'
THEN
    .....
END IF;
```

L'affectation

❑ ATTENTION :

**UN SELECT DOIT TOUJOURS AVOIR UNE VARIABLE QUI REÇOIT LE RÉSULTAT
!!! A ne pas faire !!!!**

```
CREATE or replace FUNCTION name_form(p_id int) RETURNS void AS $$  
  DECLARE  
    v_form formateur%ROWTYPE;  
  BEGIN  
    SELECT * FROM formateur  
      WHERE id_formateur=p_id ;  
    .....;  
  END; $$ LANGUAGE plpgsql;
```

Cette requête ne sert à rien : aucune variable ne reçoit le résultat et elle n'est pas utilisée dans un test logique...
Elle génère une erreur

Erreur SQL :

ERREUR: la requête n'a pas de destination pour les données résultantes

L'affectation

■ Utilisation de PERFORM

■ Exemple : pour savoir si une requête retourne une ligne

```
CREATE or replace FUNCTION name_form(p_id int) RETURNS void AS $$  
DECLARE  
BEGIN  
    PERFORM * FROM formateur  
    WHERE id_formateur = p_id ;  
  
    if FOUND then -- FOUND retourne VRAI si la requête précédente retourne au moins une ligne  
        raise exception 'il existe';  
    else  
        raise exception 'il n''existe pas';  
    end if;  
END; $$ LANGUAGE plpgsql;
```

Exécuter des commandes

■ EXECUTE chaîne-commande

chaîne-commande est une expression manipulant une chaîne (de type text) contenant la commande à exécuter.

Cette chaîne est littéralement donnée à manger au moteur SQL.

Exemple :

```
EXECUTE 'UPDATE tbl
        SET ' || quote_ident(colname) || ' = ' || quote_literal(p_name)
        || 'WHERE ... ';
```

quote_ident : contient les variables contenant les identifiants de colonne et de table

quote_literal : contient les variables contenant les valeurs de type chaînes de caractères dans la commande construite.

Elles effectuent les traitements appropriés pour renvoyer le texte entré, enfermé entre doubles ou simples guillemets respectivement, chaque caractère spécial correctement échappé

Exécuter des commandes

■ EXECUTE chaîne-commande ;

Exemple :

```
EXECUTE 'UPDATE tbl SET ' || quote_ident(colname)
        || ' = ' || quote_literal(p_name)
        || ' WHERE ... ' ;
```

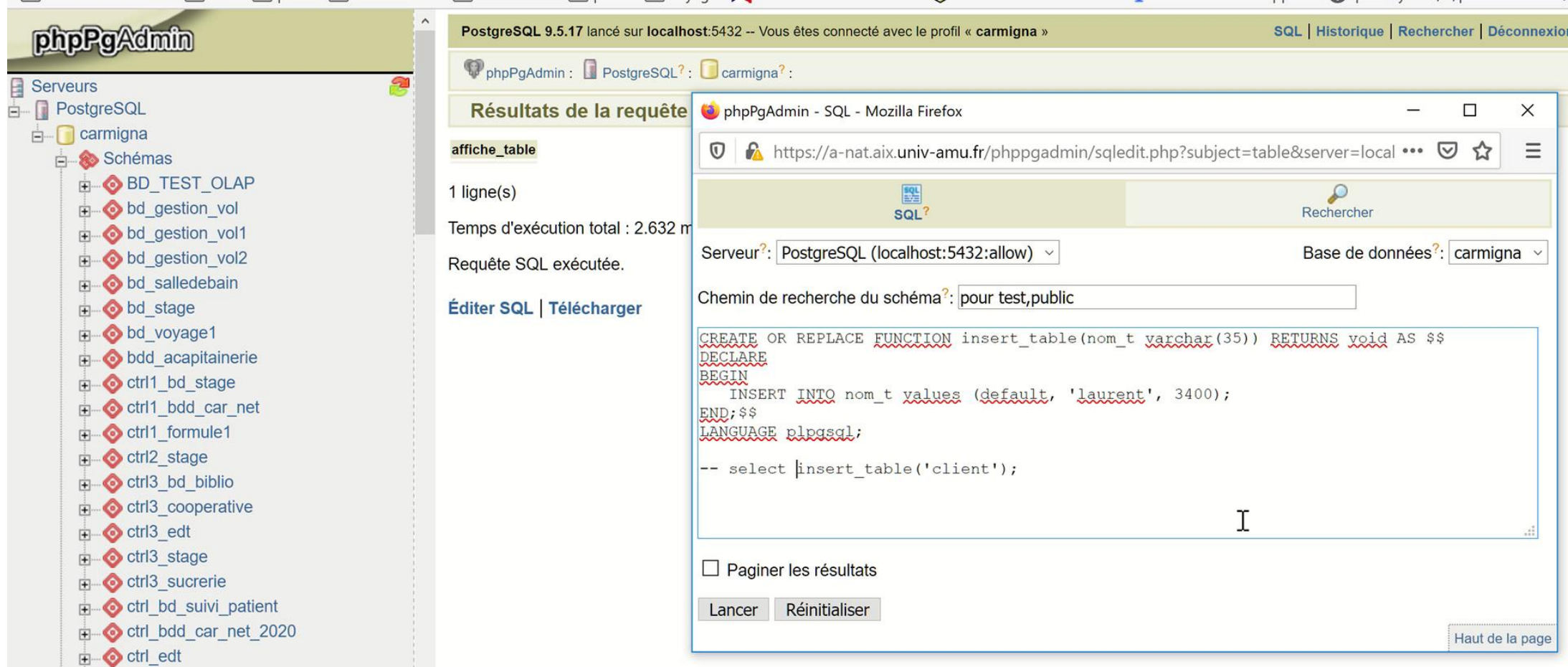
si la variable *colname* est égal à *nom_cli* et *p_name* à 'Dupond'

La chaîne de caractère va être évaluée par le EXECUTE , ce donnera :

```
UPDATE tbl
SET nom_cli= 'Dupond'
WHERE ... ;
```

La vidéo de la diapo qui suit montre l'importance du EXECUTE quand on veut passer en paramètre des noms de tables ou de colonnes

Pourquoi utiliser le EXECUTE : voir la vidéo



The screenshot displays the phpPgAdmin web interface for a PostgreSQL 9.5.17 database running on localhost:5432. The user is logged in as 'carmigna'. The left sidebar shows a tree view of the database structure, including a 'carmigna' database with several schemas and tables. The main panel shows the 'Résultats de la requête' (Query Results) for the 'affiche_table' query, indicating 1 line(s) of results and a total execution time of 2.632 ms. The 'Requête SQL exécutée' (SQL Query Executed) section shows the following SQL code:

```
CREATE OR REPLACE FUNCTION insert_table(nom_t varchar(35)) RETURNS void AS $$  
DECLARE  
BEGIN  
    INSERT INTO nom_t values (default, 'laurent', 3400);  
END;$$  
LANGUAGE plpgsql;  
  
-- select insert_table('client');
```

The interface also includes a search bar, a dropdown for the server (PostgreSQL (localhost:5432:allow)), a dropdown for the database (carmigna), and a search path (pour test,public). The 'Lancer' (Run) button is visible at the bottom of the query editor.

Structures de contrôle

■ Contrôles conditionnels :

PL/pgSQL a quatre formes de IF :

```
IF ... THEN ... END IF;  
IF ... THEN ... ELSE ... END IF;  
IF ... THEN ... ELSE IF ... END IF .. END IF;  
IF ... THEN ... ELSE IF ... THEN ... ELSE ... END IF .. END IF;
```

■ Exemple :

```
IF parentid IS NULL OR parentid = ''  
    THEN RETURN vname ;  
    ELSE RETURN parentid || '/' || vname;  
END IF;
```

Structures de contrôle

■ Contrôles conditionnels :

Le CASE WHEN END CASE;

```
CASE WHEN ...  
      THEN .....  
      ELSE .....  
END CASE;
```

■ Exemples :

```
.....  
CASE x WHEN 1, 2  
      THEN msg := 'un ou deux';  
      ELSE msg := 'autres valeurs que un ou deux';  
END CASE;  
Return msg;  
.....
```

Structures de contrôle

■ Boucles Simples

LOOP : c'est une boucle inconditionnelle répétée indéfiniment jusqu'à ce qu'elle soit terminée par une instruction EXIT ou RETURN.

■ Syntaxe :

```
[<< label >> ] LOOP instructions END LOOP;
```

■ Exemple :

```
LOOP
```

```
... traitements
```

```
IF count > 0 THEN EXIT; -- sortie de boucle
```

```
END IF;
```

```
END LOOP;
```

```
LOOP -- Autre méthode pour sortir de la boucle
```

```
... traitements
```

```
EXIT WHEN count > 0;
```

```
END LOOP;
```

Il est obligatoire d'avoir une instruction EXIT pour sortir de la boucle

Structures de contrôle

Boucles Simples

Autre Exemple en utilisant un label:

```
CREATE or REPLACE function test(x int) returns text as $$  
DECLARE v_int int := x; y int :=0;  
BEGIN  
<<loop1>>  
LOOP  
    LOOP  
        EXIT loop1 when y > 4;  
        y := y+1;  
    END LOOP;  
    v_int := v_int+1;  
END LOOP;  
RETURN 'fini = ' || v_int;  
END; $$ LANGUAGE plpgsql;
```

L'appel à la fonction :

```
select test(1);
```

Renverra :

test
fini = 1

On reste dans la deuxième boucle, tant que y est <= à 4, on sort de la boucle nommée *loop1* cad la 1^{er}, au delà de 4, on reste donc qu'une seule itération dans la 1^{er} boucle et 4 dans la 2^{ème}

Structures de contrôle

■ Boucles Simples

WHILE : L'instruction WHILE répète une séquence d'instructions aussi longtemps que l'expression conditionnelle est évaluée à vrai.

■ Syntaxe :

```
WHILE expression LOOP instructions END LOOP;
```

■ Exemple :

```
WHILE montant > 0 AND balance > 0
LOOP
    -- quelques traitements ici
END LOOP;

-- Autre exemple
WHILE NOT (prix = 0) LOOP
    -- quelques traitements ici
END LOOP;
```

Bien entendu comme dans n'importe quel autre langage vous devez faire évoluer les variables qui sont testées dans la condition pour espérer sortir de la boucle

Structures de contrôle

■ Boucles Simples :

FOR : Cette forme de FOR crée une boucle qui effectue une itération sur une plage de valeurs entières.

■ Syntaxe :

```
FOR nom IN [ REVERSE ] Debut .. Fin  
LOOP instruction END LOOP;
```

■ Exemple :

```
FOR i IN 1..10 LOOP -- progression croissante  
    RAISE NOTICE 'i is %', i;  
END LOOP;
```

```
-- Autre exemple : itération décroissante  
FOR i IN REVERSE 10..1 LOOP -- progression décroissante  
    -- quelques traitements ici  
END LOOP;
```

Structures de contrôle

■ Boucler dans le résultat d'une requête

En utilisant un type de FOR différent, vous pouvez itérer au travers des résultats d'une requête et par là même manipuler ces données.

■ Syntaxe :

```
FOR record_ou_ligne IN requête  
LOOP  
    instructions  
END LOOP;
```

- requête doit être une commande SELECT
- La requête est exécutée une seule fois
- Attention pas de point virgule(;) après la requête

Structures de contrôle

■ Boucler dans le résultat d'une requête

■ Exemple :

```
CREATE or replace FUNCTION copy_form(p_id int, p_temp varchar(20)) RETURNS
void AS $$
DECLARE use_form formateur%ROWTYPE;
BEGIN
    EXECUTE 'CREATE TABLE ' || quote_ident(p_temp)
            || '(id_f integer, nom varchar(50))';

    FOR use_form IN SELECT * FROM formateur where id_formateur < p_id
    LOOP
        EXECUTE 'INSERT INTO ' || quote_ident(p_temp) || '
                VALUES ( ' || use_form.id_formateur || ',
                        ' || quote_literal(use_form.nom_formateur) || ')';
    END LOOP;
END; $$ LANGUAGE plpgsql;
```

Structures de contrôle

■ Boucler dans les résultats d'une requête :

```
FOR record_ou_ligne IN EXECUTE expression_texte
LOOP
    instructions
END LOOP;
```

Ceci est identique à la forme précédente, à ceci près que l'expression SELECT source est spécifiée comme une expression chaîne, évaluée et exécutée à chaque entrée dans la boucle FOR.

Retour d'une fonction

- Une fonction peut retourner :
 - une valeur
`RETURN expression`
 - une liste de valeur
`RETURN NEXT expression`
 - ou le résultat d'une requête
`RETURN QUERY requête`

Retour d'une fonction

■ ATTENTION lors de l'appel d'une fonction

Il convient de distinguer deux cas pour appeler une fonction :

1. La fonction retourne une seule colonne, quelque soit les nombres de lignes

L'appel se fera

```
SELECT nomfonct(valeur paramêtres)
```

2. La fonction retourne plusieurs colonnes, quelque soit les nombres de lignes

L'appel se fera

```
SELECT *  
FROM nomfonct(valeur paramêtres) AS (col1 type, col2 type ...)
```

col1, col2 .. permettent de récupérer les valeurs retournées par la fonction

Retour d'une fonction qui retourne une seule colonne

■ La fonction retourne une seule colonne et seule ligne

■ Syntaxe générale

```
CREATE OR REPLACE FUNCTION nom_fonct() RETURNS type AS $$  
DECLARE .....  
BEGIN  
    .....  
    RETURN expression;  
END; $$ LANGUAGE plpgsql;
```

- RETURN accompagné d'une expression termine la fonction et renvoie le valeur d'expression à l'appelant
- Le type de retour de la fonction peut être :
 - Un scalaire : integer, varchar ...

Retour d'une fonction qui retourne une seule colonne

■ La fonction retourne une seule colonne et seule ligne

■ Exemple

```
CREATE OR REPLACE FUNCTION info(id int) RETURNS real AS $$  
DECLARE  
    sal real;  
BEGIN  
    SELECT salaire INTO sal FROM client WHERE id_cli=id;  
    RETURN sal;  
END; $$ LANGUAGE plpgsql;
```

-- appel de la fonction

```
SELECT info(1);  
  
SELECT * FROM client WHERE salaire > info(1);
```

Retour d'une fonction qui retourne une seule colonne

■ La fonction retourne une seule colonne et plusieurs lignes

■ Syntaxe générale

```
CREATE OR REPLACE FUNCTION nom_fonct() RETURNS SETOF type AS $$  
DECLARE .....  
BEGIN  
    .....  
    RETURN NEXT expression;  
    .....  
    RETURN;  
END; $$ LANGUAGE plpgsql;
```

- Lorsqu'une fonction PL/pgSQL est déclarée pour renvoyer un *SETOF type* quelconque, la procédure à suivre est légèrement différente. Dans ce cas, les items individuels à renvoyer sont spécifiés dans les commandes **RETURN NEXT**, et ensuite une commande RETURN finale, sans arguments est utilisée pour indiquer que la fonction a terminé son exécution, ce dernier peut être omis!
- RETURN NEXT peut renvoyer des types scalaires et des types composites de données;

Retour d'une fonction qui retourne une seule colonne

■ La fonction retourne une seule colonne et plusieurs lignes

■ Exemple

```
CREATE OR REPLACE FUNCTION info(id int) RETURNS SETOF real AS $$
DECLARE    sal real ;
BEGIN
    FOR sal IN SELECT salaire FROM client
        WHERE salaire >= (SELECT salaire FROM client WHERE id_cli=id)
    LOOP    RETURN NEXT sal;
    END LOOP;
END; $$ LANGUAGE plpgsql;

-- appel de la fonction
SELECT info(1);
SELECT * FROM client WHERE salaire >= all( select info(1));
```

Retour d'une fonction qui retourne plusieurs colonnes

■ La fonction retourne **plusieurs colonnes et seule ligne**

■ Syntaxe générale

```
CREATE OR REPLACE FUNCTION nom_fonct() RETURNS record AS $$  
DECLARE .....  
BEGIN  
    .....  
    RETURN record;  
END; $$ LANGUAGE plpgsql;
```

- Le type de retour de la fonction peut être :
 - Un type composite: un record

Retour d'une fonction qui retourne plusieurs colonnes

■ La fonction retourne plusieurs colonnes et seule ligne

■ Exemple

```
CREATE OR REPLACE FUNCTION info(id int) RETURNS record AS $$  
DECLARE info record ;  
BEGIN  
    SELECT nom,salaire INTO info FROM client WHERE id_cli=id;  
    RETURN info ;  
END; $$ LANGUAGE plpgsql;
```

-- appel de la fonction

```
SELECT * FROM info(1) AS (n varchar, s real);
```

n et s vont récupérer respectivement les valeurs de nom et salaire

n	s
bidule	3300

-- On peut afficher que l'une des deux colonnes :

```
SELECT n FROM info(1) AS (n varchar, s real);
```

n
bidule

Retour d'une fonction qui retourne plusieurs colonnes

■ La fonction retourne plusieurs colonnes et plusieurs lignes

■ Syntaxe générale

```
CREATE OR REPLACE FUNCTION nom_fonct() RETURNS SETOF record AS $$  
DECLARE .....  
BEGIN  
    .....  
    RETURN NEXT expression;  
    .....  
    RETURN;  
END; $$ LANGUAGE plpgsql;
```

- Le type de retour de la fonction peut être :
 - Un type composite: un record

Retour d'une fonction qui retourne plusieurs colonnes

■ La fonction retourne plusieurs colonnes et plusieurs lignes

■ Exemple

```
CREATE OR REPLACE FUNCTION info(id int) RETURNS SETOF record AS $$  
DECLARE    info client%ROWTYPE;  
BEGIN  
    FOR info IN SELECT * FROM client  
        WHERE salaire >= (SELECT salaire FROM client WHERE id_cli=id)  
    LOOP    RETURN NEXT info;  
    END LOOP;  
END; $$ LANGUAGE plpgsql;
```

-- appel de la fonction

```
SELECT * FROM info(1) AS (i int, n varchar, s real);
```

Retour d'une fonction *(en utilisant des paramètre en sortie)*

■ Fonctions SQL avec des paramètres en sortie

- Une autre façon de décrire les résultats d'une fonction est de la définir avec des paramètres en sortie comme dans cet exemple :

```
CREATE OR REPLACE FUNCTION somme (IN x numeric, IN y numeric , OUT somme
numeric) AS $$
BEGIN
    somme := x + y ;
END; $$ LANGUAGE plpgsql;
```

- **Attention** définir un paramètre comme paramètre de sortie, implique de ne pas mettre de RETURN dans la fonction et pas de RETURNS type avant le AS
- L'appel à la fonction sera ***select somme(4,7)*** par exemple
- Vous pouvez omettre le IN devant les paramètres d'entrée (par défaut en entrée)

Erreur à ne pas faire : voir la vidéo

The screenshot displays the phpPgAdmin web interface for PostgreSQL 9.5.17 running on localhost:5432. The user is connected as 'carmigna'. The left sidebar shows a tree view of the database structure, including tables, views, sequences, and functions. The main panel shows the 'Fonction' (Function) section with a list of functions and their return types. A modal window titled 'phpPgAdmin - SQL - Mozilla Firefox' is open, showing the SQL editor for the 'pour test' database. The editor contains a PL/SQL function definition for 'info' and a 'SELECT info(1)' query. The function is defined as follows:

```
CREATE OR REPLACE FUNCTION info(id int) RETURNS SETOF record AS $$  
DECLARE  
    info client%ROWTYPE;  
BEGIN  
    FOR info IN SELECT * FROM client  
        WHERE salaire >= (SELECT salaire FROM client WHERE id_cli=id)  
    LOOP  
        RETURN NEXT info;  
    END LOOP;  
END; $$ LANGUAGE plpgsql;  
  
SELECT info(1)
```

The modal window also includes a search bar, a dropdown for the server (PostgreSQL (localhost:5432:allow)), a dropdown for the database (carmigna), and a dropdown for the schema (pour test,public). The 'Lancer' (Run) button is visible at the bottom of the modal.

Messages et erreurs

■ L'instruction RAISE permet de renvoyer différent niveau de message

■ Syntaxe :

```
RAISE niveau 'texte', [expr[,expr]]
```

niveau ∈ {DEBUG, LOG, INFO, NOTICE, WARNING, EXCEPTION}

texte est une chaîne où % est remplacé par l'expression (expr) correspondante.

■ Exemple :

```
RAISE EXCEPTION 'taux négatif % et prix = %', taux, prix;
```

La génération d'une exception interrompt la transaction en cours

Gestion des erreurs

■ Récupération des erreurs

```
[ <<label>> ]  
[ DECLARE declarations ]  
BEGIN  
instructions  
EXCEPTION WHEN condition [ OR condition ... ]  
THEN instructions_gestion_erreurs  
[ WHEN condition [ OR condition ... ] THEN  
instructions_gestion_erreurs ... ]  
END;
```

Voir les [Codes d'erreurs de PostgreSQL](#).

Gestion des erreurs

■ Exemple

```
CREATE OR REPLACE FUNCTION formateurs(p_diviseur int) RETURNS void AS $$
DECLARE
    x integer :=30;
    y integer;
BEGIN
    y := x / p_diviseur ;
    INSERT INTO formateur VALUES (3,'Morel','Paul',45);
EXCEPTION
    WHEN division_by_zero THEN /* erreur possible sur la division */
        RAISE EXCEPTION 'Erreur division_by_zero';
    WHEN unique_violation THEN /* erreur possible sur l'INSERT */
        UPDATE formateur
        SET (nom_form, prenom_form, nbh)=( 'Morel','Paul',45)
        WHERE id_formateur=3;
RETURN ;
END; $$ LANGUAGE plpgsql;
```

Gestion des erreurs

■ 1^{er} appel

```
SELECT formateurs (3) ;
```

Mise à jour du formateur de la table formateur

■ 2^{ème} appel

```
SELECT formateurs (0) ;
```

La division par zéro déclenche l'exception

Le INSERT INTO formateur ... ne sera pas exécuté

Erreur SQL :

ERREUR: Erreur division_by_zero

Dans l'instruction :
Select formateurs(0) ;



FIN